

Heaps

Heap : A (binary) heap data structure is an array-based object that can be viewed as a nearly complete binary tree

- An array $A[1:n]$ represents a heap

↳ $A.\text{heap-size}$ represents the number of elements in the heap

$$0 \leq A.\text{heap-size} \leq n$$

and

$A[1 : A.\text{heap-size}]$ holds the set of valid elements.

- Root is at $A[1]$

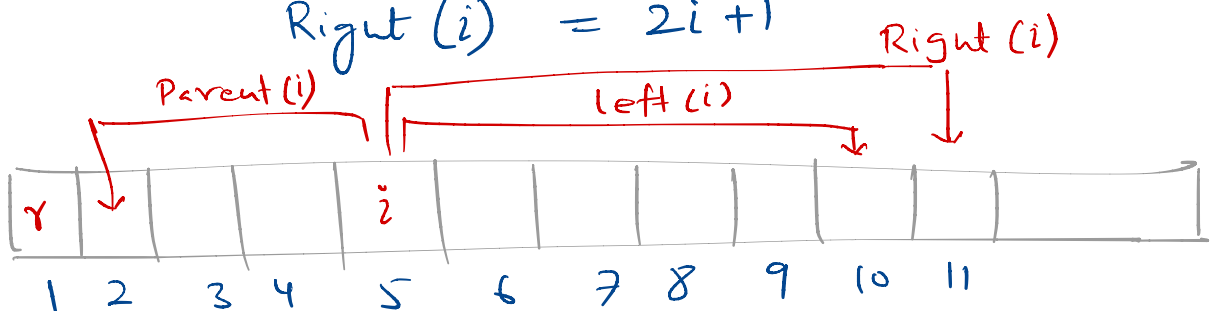
- If $A.\text{heap-size} == 0$ then the heap is empty.

- Given an index i of a node

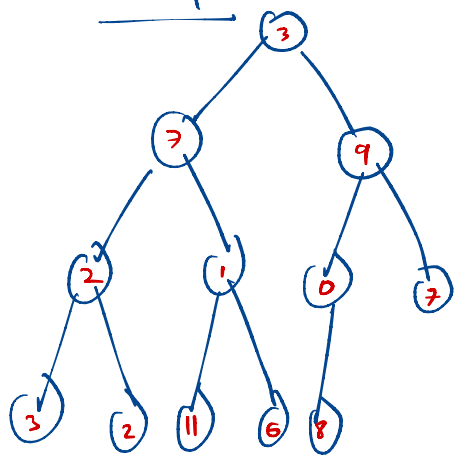
$$\text{Parent}(i) = \lfloor i/2 \rfloor$$

$$\text{Left}(i) = 2i$$

$$\text{Right}(i) = 2i + 1$$



Example :



3	7	9	2	1	0	7	3	2	11	6	8			
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

- In a max-heap, the max-heap property is satisfied at every node i

$$A[\text{parent}(i)] \geq A[i]$$

- In a min-heap, the min-heap property is satisfied at every node i

$$A[\text{parent}(i)] \leq A[i]$$

- Height of a node in a heap is computed similar to trees, as :

of edges on the longest simple downward path from node to leaf.

↪ Height of a heap = height of the root.

Max-Heapify : maintains / re-establishes
the max-heap property

Max-Heapify (A, i) $\rightarrow$ assumes that subtrees
at $\text{Left}(i)$ & $\text{Right}(i)$
are max-heaps
and $A[i]$ violates the max-heap
property

Max-Heapify (A, i)

$l = \text{Left}(i)$

$r = \text{Right}(i)$

if $l \leq A.\text{heap-size}$ & $A[l] > A[i]$
 $\text{largest} = l$

else $\text{largest} = i$

if $r \leq A.\text{heap-size}$ & $A[r] > A[\text{largest}]$
 $\text{largest} = r$

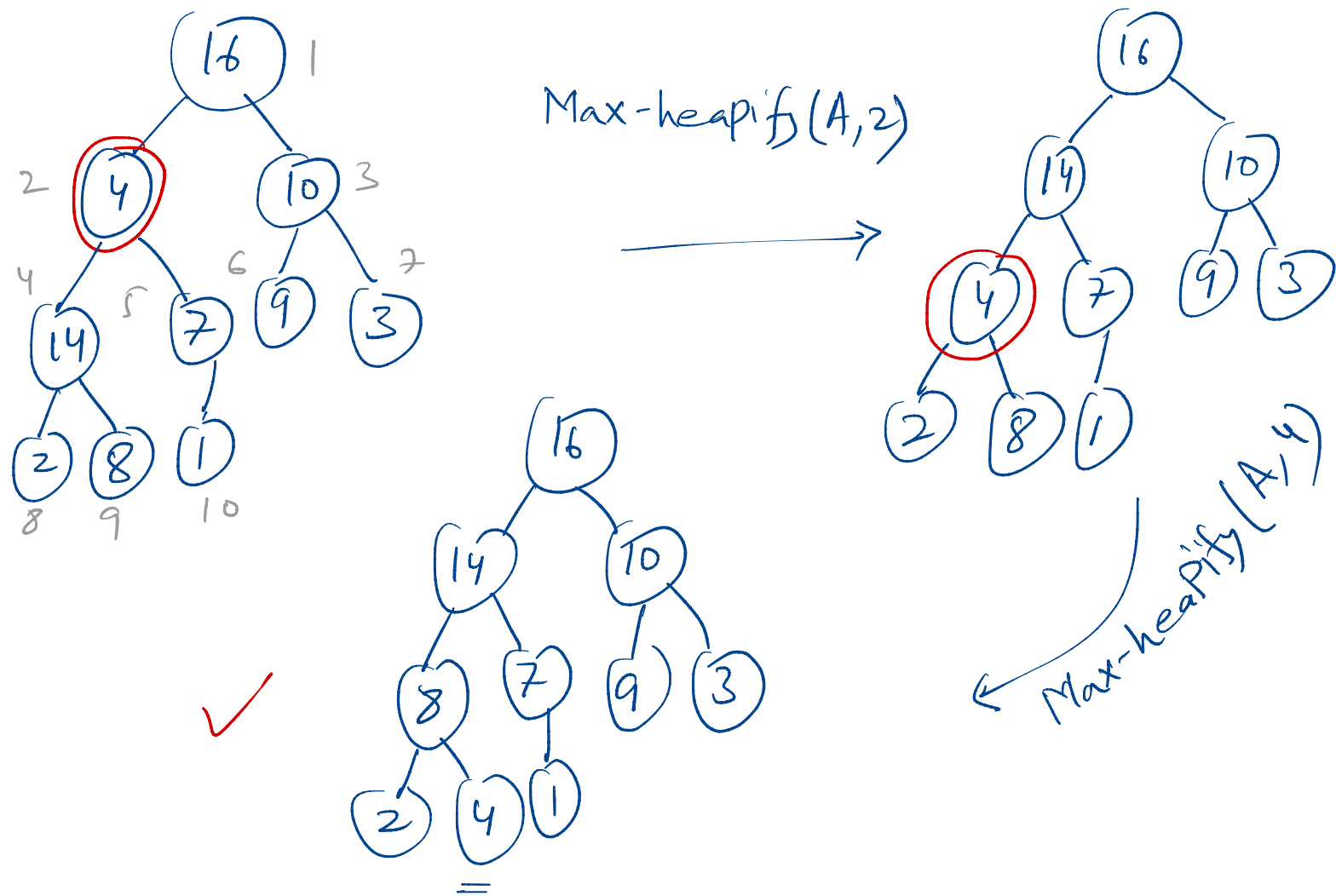
if $\text{largest} \neq i$

 exchange $A[i]$ with $A[\text{largest}]$

 Max-heapify ($A, \text{largest}$)

$O(\lg n)$

Example :

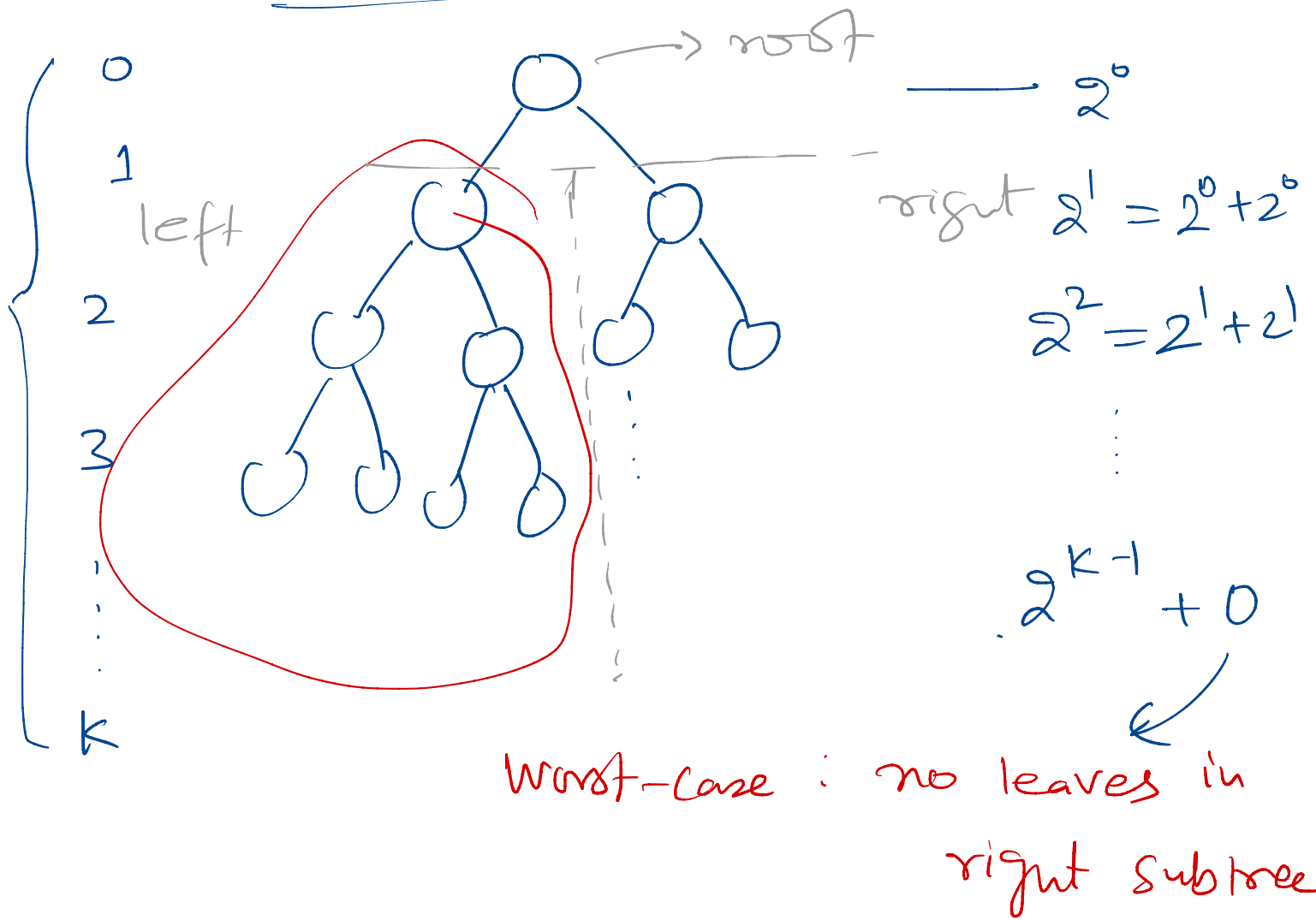


Complexity analysis :

$$T(n) \leq T\left(\frac{2n}{3}\right) + \Theta(1)$$

$$\Rightarrow T(n) = O(\lg n)$$

Sub-problem size :



$$n_{\text{left}} = \sum 2^0 + 2^1 + \dots + 2^{k-1}$$

$$= 2^k - 1$$

$$n_{\text{right}} = 2^{k-1} - 1$$

$$n_{\text{tot}} = 1 + n_{\text{left}} + n_{\text{right}}$$

$$= 1 + n_{\text{left}} + 2^{k-1} - 1$$

$$= n_{\text{left}} + \frac{(2^k - 1 + 1)}{2} =$$

$$= n_{\text{left}} + \frac{n_{\text{left}} + 1}{2}$$

$$n_{\text{tot}} = \frac{3n_{\text{left}} + 1}{2}$$

$$\Rightarrow n_{\text{left}} = \frac{2n_{\text{tot}} - 1}{3}$$

$$n_{\text{left}} < \frac{2n_{\text{tot}}}{3}$$

Building a max-heap

↳ given an array $A[1:n]$

↳ Bottom up approach

Note: Subarray $A[\lfloor \frac{n}{2} \rfloor + 1 : n]$
are all leaves

⇒ single-node max
heaps!

Build - Max - Heap (A, n)

A: heap-size = n

for $i = \lfloor \frac{n}{2} \rfloor$ ↗ last non-leaf node down to 1

Max-Heapify (A, i)

↙

$O(n)$

Heap-Sort

Given input array $A[1:n]$
Sort in ascending order.

HeapSort(A, n)

Build-Max-Heap(A, n)

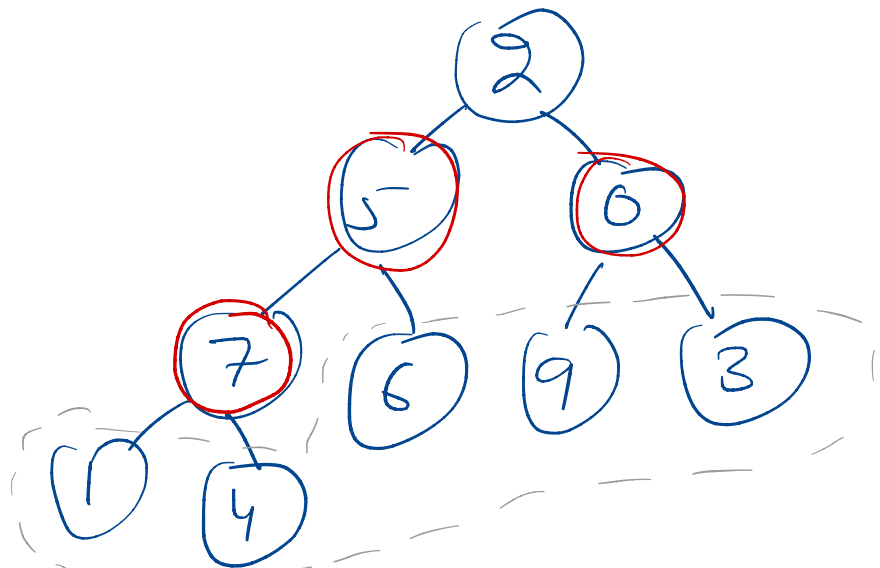
for $i = n$ downto 2

 Exchange $A[1]$ with $A[i]$
 $A.\text{heap-size} = A.\text{heap-size} - 1$
 Max-Heapify($A, 1$)

Complexity : $T(n) = \overset{\text{Build}}{O(n)} + (n-1) O(\lg n)$
 $= O(n \lg n)$

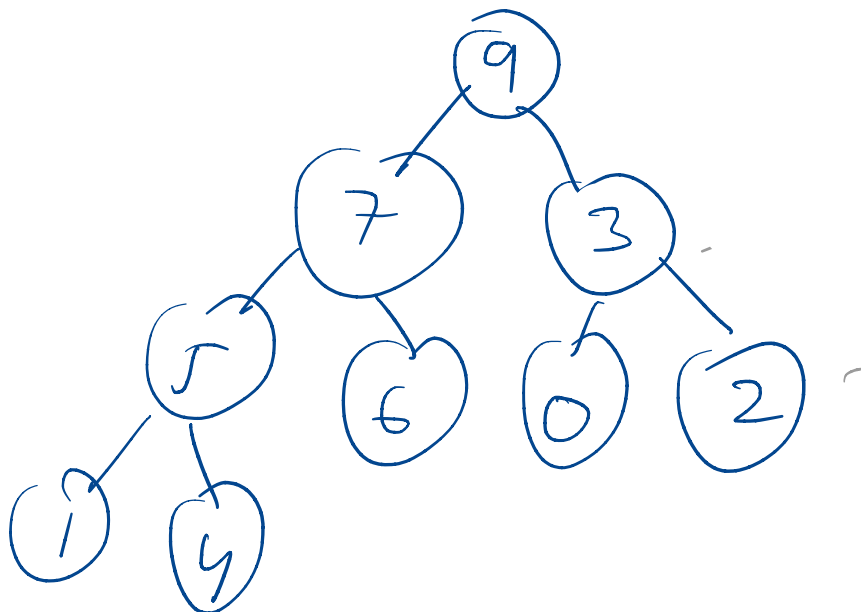
Example

2	5	0	7	6	9	3	1	4
1	2	3	4	5	6	7	8	9

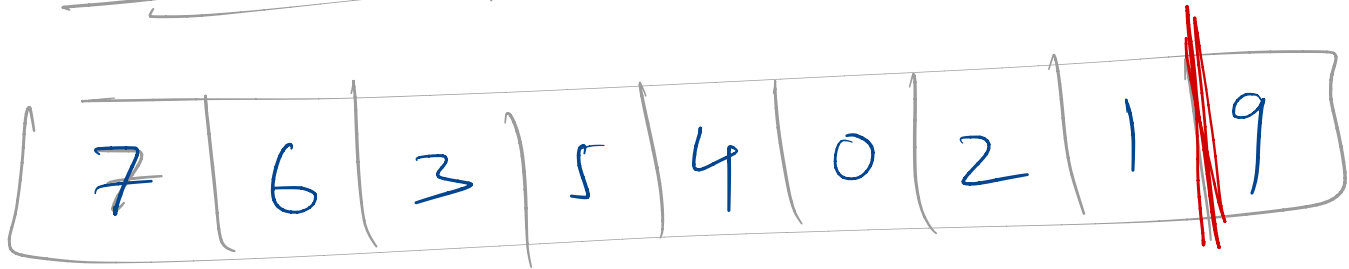
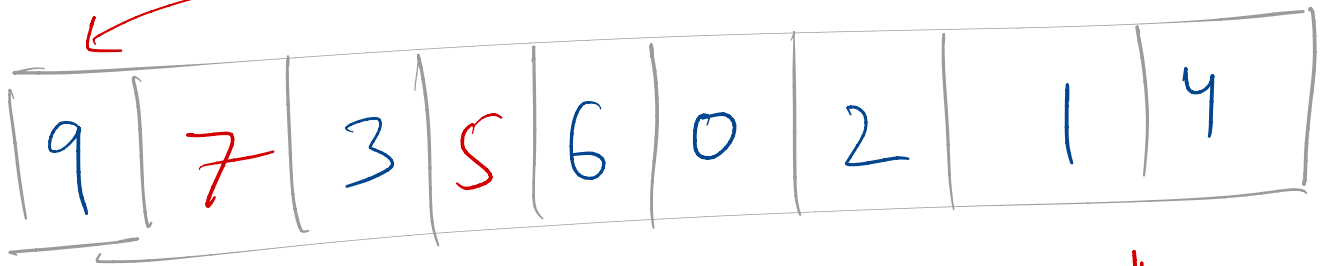


① Build - Max-Heap (A, n)

9	7	3	5	6	0	2	1	4
---	---	---	---	---	---	---	---	---



② Max-heapify on $n \rightarrow 2$



0
6
0

